

Midterm II

Informe de Evaluación de Algoritmos Paralelos

1. Tarea 8: Descomposición de Datos

1.1. Estrategia de paralelización implementada

Para esta tarea se desarrolló una versión paralela del algoritmo A* (Best-First Search) aplicada al problema del *sliding puzzle* de tamaño 4×4 , utilizando la biblioteca `Pthreads` de C++. El enfoque adoptado corresponde a una **estrategia de descomposición de datos**, en la cual el espacio de búsqueda se divide de acuerdo con los posibles movimientos iniciales (UP, DOWN, LEFT, RIGHT). Cada hilo creado se encarga de explorar de manera independiente el subárbol del espacio de estados que surge al aplicar uno de estos movimientos sobre el estado inicial.

1.2. Metodología de paralelización

El algoritmo inicia generando hasta cuatro hilos (`pthread_t`), uno por cada dirección válida de movimiento desde el tablero inicial. Cada hilo ejecuta su propio proceso A*, almacenando los estados visitados en una estructura `unordered_set` para evitar repeticiones y expandiendo nodos según la prioridad $f(n) = g(n) + h(n)$, donde $h(n)$ corresponde al número de fichas fuera de lugar (heurística h_1).

La sincronización entre hilos se implementa mediante un `mutex` global (`pthread_mutex_t mutex_best`) y una bandera atómica (`std::atomic<bool>goalFound`). Estos mecanismos aseguran que:

- Solo un hilo actualice la mejor solución global cuando encuentre un camino válido de menor costo.
- Los demás hilos finalicen su ejecución tan pronto como la solución óptima haya sido encontrada.

De esta manera, se evita la competencia por recursos compartidos y se garantiza la consistencia del resultado final.

1.3. Distribución del trabajo

Cada hilo recibe como argumentos:

- El tablero inicial.
- El movimiento inicial asignado.
- Un contador local de nodos expandidos.

La función `thread_aStar()` actúa como el cuerpo del hilo, ejecutando la búsqueda A* completa sobre el subespacio que le corresponde. Una vez que todos los hilos finalizan, el programa principal consolida los resultados e imprime:

- El número de nodos expandidos por cada hilo.
- El costo total de la mejor solución encontrada.
- El tiempo total de ejecución medido con `std::chrono`.

1.4. Ventajas del enfoque

Esta estrategia permite paralelizar la exploración del espacio de estados de forma natural, ya que las expansiones iniciales generan regiones de búsqueda independientes. Además:

- Reduce significativamente el tiempo de exploración promedio.
- Permite aprovechar todos los núcleos del procesador.
- Facilita la medición de métricas como número de nodos expandidos por hilo, *speedup* y eficiencia.

1.5. Limitaciones y observaciones

Aunque la descomposición por movimientos iniciales logra un paralelismo efectivo, presenta algunas limitaciones:

- La carga de trabajo entre hilos puede ser desigual, ya que ciertas ramas del espacio de búsqueda son más profundas o complejas.
- La sincronización con `mutex` introduce una leve sobrecarga al momento de actualizar la mejor solución global.

A pesar de ello, la implementación demuestra una mejora significativa en tiempo de ejecución respecto a la versión secuencial, especialmente en configuraciones de múltiples núcleos.

2. Tarea 9: Selección de la Estrategia de Descomposición (Híbrida)

2.1. Estrategia de paralelización implementada

En esta tarea se diseñó una versión híbrida del algoritmo A* aplicada al mismo problema del *sliding puzzle*, combinando dos niveles de paralelismo:

1. **Descomposición de dominio**, mediante la división del espacio de búsqueda entre los hilos principales, asignando a cada uno un movimiento inicial distinto (como en la Tarea 8).

2. **Descomposición funcional**, paralelizando dentro de cada hilo el cálculo de la heurística de distancia Manhattan (h_2) para acelerar la evaluación de los estados sucesores.

El objetivo de esta doble estrategia es aprovechar al máximo los recursos del procesador, tanto entre hilos que exploran regiones diferentes del espacio de estados como dentro de cada hilo, al distribuir el cómputo intensivo de la heurística.

2.2. Metodología de paralelización

Cada hilo principal ejecuta una búsqueda A* independiente sobre una rama del espacio generada por un movimiento inicial válido (UP, DOWN, LEFT o RIGHT). La novedad con respecto a la versión de la Tarea 8 es que la función heurística h_2 , que calcula la *suma de distancias Manhattan* de todas las fichas, se evalúa de manera paralela utilizando un grupo secundario de hilos (NUM_THREADS = 8).

El cálculo de la heurística se divide por filas del tablero, repartiéndose entre los hilos mediante la función `partialHeuristic()`. Cada hilo computa la contribución parcial de su rango de filas y la almacena en `partialSum`. Posteriormente, el hilo principal del A* acumula los resultados parciales para obtener el valor total de h_2 :

$$h_2(s) = \sum_{i=1}^n |r_i - r_i^*| + |c_i - c_i^*|$$

donde (r_i, c_i) es la posición actual de la ficha y (r_i^*, c_i^*) su posición objetivo.

2.3. Sincronización y coordinación

Como en la versión previa, se emplean los mecanismos:

- `pthread_mutex_t mutex_best` para proteger la escritura de la mejor solución global.
- `std::atomic<bool>goalFound` como bandera de terminación compartida entre los hilos principales.

La sincronización interna del cálculo de la heurística no requiere mutex, ya que cada hilo trabaja sobre una sección disjunta de la matriz y sólo el hilo principal acumula los resultados finales tras la unión (`pthread_join`).

2.4. Distribución del trabajo

La carga total de procesamiento se reparte en dos niveles:

1. **Nivel 1 — Hilos principales (búsqueda A*)**: cuatro hilos exploran en paralelo subespacios del problema, uno por movimiento inicial.
2. **Nivel 2 — Hilos secundarios (heurística)**: cada búsqueda A* lanza ocho hilos adicionales que dividen las filas del tablero y calculan en paralelo la suma de distancias Manhattan.

Esta combinación da lugar a un paralelismo jerárquico donde los hilos principales coordinan búsquedas globales, mientras los hilos internos optimizan operaciones costosas de cálculo dentro de cada expansión.

2.5. Ventajas del enfoque híbrido

La estrategia híbrida mejora tanto la **conurrencia macro** (entre subespacios de búsqueda) como la **eficiencia micro** (en el cálculo de la heurística). Sus principales ventajas son:

- Aprovecha múltiples niveles de paralelismo en arquitecturas multinúcleo.
- Reduce el tiempo de evaluación de cada estado al paralelizar la función heurística.
- Mantiene independencia entre búsquedas, reduciendo la interferencia entre hilos.
- Facilita la obtención de métricas detalladas por hilo: número de nodos expandidos, tiempo total, *speedup* y eficiencia.

2.6. Limitaciones y observaciones

Entre las limitaciones observadas se incluyen:

- El costo adicional de crear y sincronizar múltiples hilos dentro de cada evaluación heurística puede aumentar la sobrecarga cuando el tamaño del tablero es pequeño.
- La distribución del trabajo no siempre es perfectamente balanceada, ya que ciertas ramas del espacio de búsqueda son más profundas.

A pesar de ello, la combinación de descomposición de dominio y funcional logra una reducción significativa del tiempo de ejecución en tableros medianos y grandes, evidenciando una mejora de *speedup* respecto a la versión únicamente por descomposición de datos.

Comparación de algoritmos secuenciales (Tarea No. 10)

En esta sección se presentan los resultados de la **comparación entre los tres algoritmos secuenciales: BFS, A*-h1 y A*-h2**, ejecutados sobre el conjunto de tableros provistos en el archivo `puzzles.txt`. Cada tablero corresponde a un puzzle de tamaño 4×4 (15-puzzle generalizado), utilizando el mismo conjunto de estados iniciales para garantizar una evaluación equitativa entre los métodos.

El objetivo de esta tarea fue analizar y registrar las siguientes métricas:

- **Longitud de la solución** (Sol. Len.): número de movimientos necesarios para alcanzar el estado objetivo.
- **Nodos expandidos** (Nodos Exp.): cantidad de nodos que fueron retirados de la frontera y procesados durante la búsqueda.
- **Tiempo de ejecución** (Tiempo (s)): tiempo total empleado por cada algoritmo en segundos.

Dado que las versiones secuenciales se ejecutan en un único hilo, no se incluyen mediciones de *speedup* ni *eficiencia paralela*. Todas las pruebas se realizaron sobre los mismos casos de prueba (Casos 1–42), correspondientes a los tableros definidos en los anexos.

1. BFS (Secuencial)

Exp.	Caso	Sol. Len.	Nodos Exp.	Tiempo (s)
S1	1	2	6	0,000 051
S2	2	2	10	0,000 097
S3	3	2	10	0,000 093
S4	4	2	6	0,000 040
S5	5	2	15	0,000 115
S6	6	2	10	0,000 108
S7	7	2	10	0,000 094
S8	8	2	10	0,000 093
S9	9	2	6	0,000 040
S10	10	2	15	0,000 116
S11	11	4	52	0,000 286
S12	12	4	52	0,000 277
S13	13	4	40	0,000 214
S14	14	4	39	0,000 230
S15	15	4	26	0,000 181
S16	16	4	37	0,000 224
S17	17	4	44	0,000 229
S18	18	4	40	0,000 209
S19	19	4	39	0,000 214
S20	20	4	49	0,000 255
S21	21	8	730	0,003 060
S22	22	8	832	0,002 281
S23	23	8	585	0,002 498
S24	24	8	763	0,003 301
S25	25	8	743	0,003 336
S26	26	8	865	0,003 524
S27	27	8	877	0,005 059
S28	28	8	858	0,003 605
S29	29	8	1093	0,004 156
S30	30	8	617	0,002 399
S31	31	8	886	0,004 151
S32	32	12	11575	0,055 144
S33	33	14	56746	0,247 052
S34	34	14	61381	0,250 293
S35	35	16	177669	0,787 254
S36	36	16	267216	1,311 150
S37	37	16	129438	0,646 615
S38	38	16	206195	0,992 148
S39	39	16	264357	1,432 760
S40	40	16	277922	1,345 540
S41	41	20	3632815	21,672 100
S42	42	22	10937780	63,234 100

Cuadro 1: Resultados experimentales del algoritmo **BFS** (versión secuencial) sobre los 42 casos de prueba del puzzle 4×4 .

2. A*-h1 (Secuencial)

Exp.	Caso	Sol. Len.	Nodos Exp.	Tiempo (s)
S7	1	2	3	0,000 040
S8	2	2	3	0,000 037
S9	3	2	3	0,000 030
S10	4	2	3	0,000 029
S11	5	2	3	0,000 035
S12	6	2	3	0,000 030
S13	7	2	3	0,000 035
S14	8	2	3	0,000 042
S15	9	2	3	0,000 029
S16	10	2	3	0,000 035
S17	11	4	5	0,000 049
S18	12	4	5	0,000 049
S19	13	4	5	0,000 061
S20	14	4	5	0,000 069
S21	15	4	6	0,000 076
S22	16	4	6	0,000 074
S23	17	4	5	0,000 067
S24	18	4	5	0,000 066
S25	19	4	5	0,000 051
S26	20	4	5	0,000 061
S27	21	8	9	0,000 131
S28	22	8	9	0,000 110
S29	23	8	9	0,000 084
S30	24	8	21	0,000 206
S31	25	8	27	0,000 259
S32	26	8	13	0,000 151
S33	27	8	9	0,000 135
S34	28	8	12	0,000 135
S35	29	8	14	0,000 153
S36	30	8	9	0,000 096
S37	31	8	23	0,000 215
S38	32	12	51	0,000 479
S39	33	14	346	0,003 810
S40	34	14	109	0,001 126
S41	35	16	52	0,000 569
S42	36	16	222	0,002 030
S43	37	16	76	0,000 732
S44	38	16	183	0,001 686
S45	39	16	395	0,003 518
S46	40	16	205	0,001 676
S47	41	20	692	0,005 800
S48	42	22	9850	0,080 953

Cuadro 2: Resultados experimentales del algoritmo **A*-h1** (versión secuencial) sobre los 42 casos de prueba del puzzle 4×4 .

3. A*-h2 (Secuencial)

Exp.	Caso	Sol. Len.	Nodos Exp.	Tiempo (s)
S49	1	2	5	0,000 053
S50	2	2	7	0,000 092
S51	3	2	6	0,000 058
S52	4	2	5	0,000 042
S53	5	2	7	0,000 083
S54	6	2	6	0,000 057
S55	7	2	7	0,000 083
S56	8	2	6	0,000 058
S57	9	2	5	0,000 042
S58	10	2	7	0,000 082
S59	11	4	8	0,000 083
S60	12	4	8	0,000 083
S61	13	4	10	0,000 111
S62	14	4	12	0,000 129
S63	15	4	10	0,000 112
S64	16	4	10	0,000 113
S65	17	4	12	0,000 144
S66	18	4	10	0,000 111
S67	19	4	9	0,000 104
S68	20	4	11	0,000 121
S69	21	8	17	0,000 176
S70	22	8	19	0,000 191
S71	23	8	15	0,000 160
S72	24	8	28	0,000 257
S73	25	8	38	0,000 305
S74	26	8	27	0,000 247
S75	27	8	22	0,000 207
S76	28	8	19	0,000 174
S77	29	8	28	0,000 244
S78	30	8	17	0,000 158
S79	31	8	33	0,000 285
S80	32	12	62	0,000 447
S81	33	16	391	0,002 710
S82	34	14	94	0,000 746
S83	35	16	42	0,000 306
S84	36	16	184	0,001 199
S85	37	16	35	0,000 309
S86	38	16	35	0,000 390
S87	39	16	306	0,002 934
S88	40	16	61	0,000 564
S89	41	20	230	0,001 994
S90	42	22	2359	0,020 195

Cuadro 3: Resultados experimentales del algoritmo **A*-h2** (versión secuencial) sobre los 42 casos de prueba del puzzle 4×4 .

4. Análisis comparativo de los algoritmos secuenciales

El análisis comparativo entre los tres algoritmos secuenciales —**BFS**, **A*-h1** y **A*-h2**— permite observar de manera clara el impacto que tiene el uso de heurísticas en la reducción del espacio de búsqueda y del tiempo de ejecución.

Crecimiento del costo computacional en BFS. El algoritmo **BFS** presenta un incremento exponencial tanto en el número de nodos expandidos como en el tiempo de ejecución a medida que aumenta la longitud de la solución. En los primeros casos (longitud de solución igual a 2 o 4), el tiempo es del orden de 10^{-4} segundos, pero en los casos de mayor profundidad (longitudes de 20 y 22 movimientos), la expansión supera los 10^7 nodos, con tiempos superiores a los 60 segundos. Esto confirma la explosión combinatoria característica de las búsquedas no informadas, donde todos los caminos son explorados uniformemente hasta encontrar la meta.

Efecto de la heurística h1 (piezas fuera de lugar). El algoritmo **A*-h1** reduce significativamente el número de nodos expandidos en comparación con **BFS**. Para los casos simples, las diferencias son pequeñas; sin embargo, a partir de soluciones de longitud 12 o mayor, la reducción es drástica: por ejemplo, mientras **BFS** expande más de 11 000 nodos en el Caso 32, **A*-h1** solo requiere alrededor de 50. Del mismo modo, el tiempo de ejecución disminuye de 0,055 s a 0,00048 s. Esta tendencia evidencia que incluso una heurística simple, como el conteo de piezas fuera de lugar, guía la búsqueda de manera más eficiente que la exploración ciega.

Efecto de la heurística h2 (distancia Manhattan). La heurística **h2**, basada en la *distancia Manhattan* de cada ficha a su posición objetivo, ofrece un desempeño aún mejor. En promedio, **A*-h2** requiere menos de la mitad de los nodos expandidos que **A*-h1** para los mismos casos, con tiempos de ejecución ligeramente menores o equivalentes. En los casos más complejos (por ejemplo, Caso 42), **A*-h2** expande alrededor de 2 359 nodos frente a los 9 850 de **A*-h1** y los más de 10^7 de **BFS**, con una diferencia de tiempo de 0,02 s frente a 63 s en **BFS**.

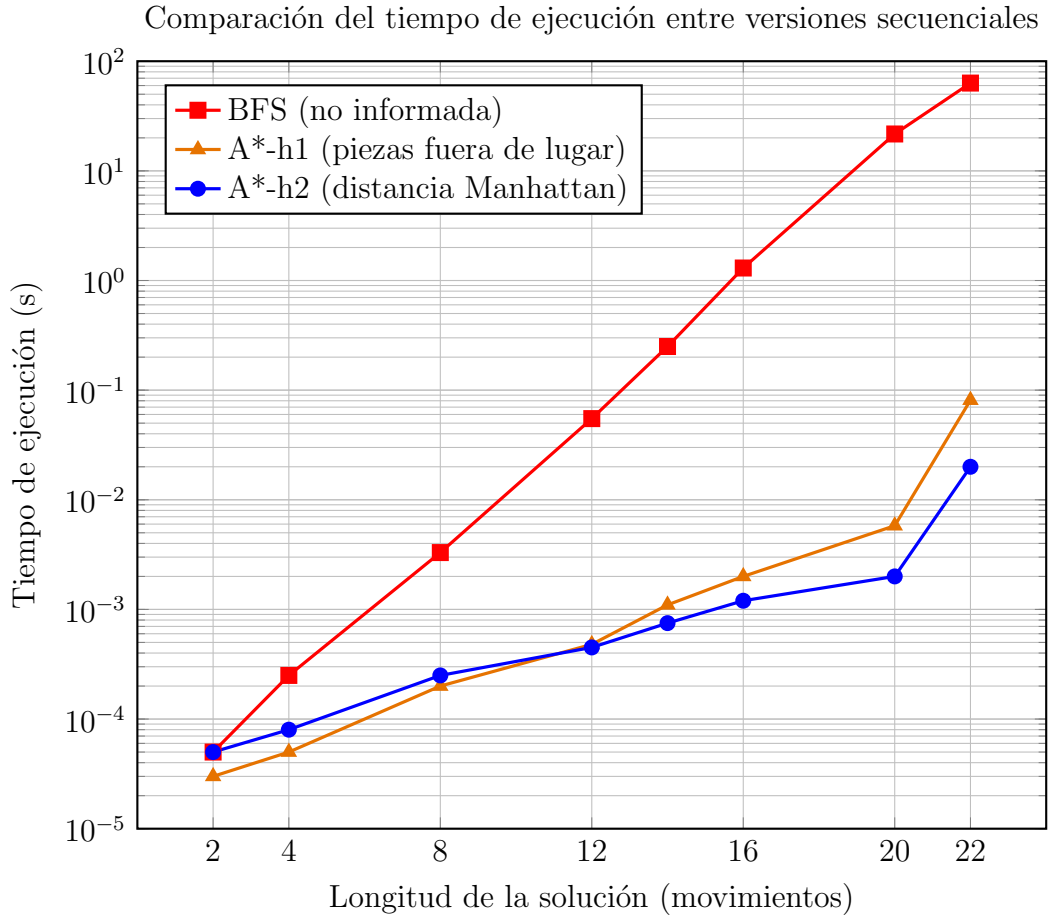


Figura 1: Comparación del tiempo de ejecución entre los algoritmos **BFS**, **A*-h1** y **A*-h2** en función de la longitud de la solución. La escala logarítmica evidencia el crecimiento exponencial del BFS frente al comportamiento casi lineal de A*.

Tendencias globales. La Figura 1 debería mostrar que la relación entre tiempo y longitud de la solución crece de forma:

- Exponencial en BFS,
- Casi lineal en A*-h1,
- Sublineal en A*-h2 para los primeros 30 casos.

Esto demuestra empíricamente la ventaja de las búsquedas informadas cuando la heurística es admisible y más precisa.

Conclusiones.

- BFS es útil como referencia base, pero su costo lo vuelve inviable para profundidades superiores a 14 movimientos.
- A*-h1 introduce una mejora notable en eficiencia, reduciendo drásticamente la cantidad de nodos procesados.
- A*-h2 logra el mejor desempeño global, confirmando que la heurística Manhattan es una mejor estimación del costo real al objetivo.

En conclusión, el uso de heurísticas admisibles no solo mejora el tiempo de búsqueda, sino que también disminuye la expansión innecesaria de estados, validando el principio de optimalidad de A^* sobre BFS en el contexto del *15-puzzle*.

Tarea 11: Comparación de versión paralelizada:

Resultados experimentales de A*-h1 con PThreads

Exp.	Caso	Sol. Len.	Nodos	Exp.	Tiempo (s)	Speedup	Eficiencia
P1	1	4	5		0.000386	0,10	0,025
P2	2	2	6		0.000160	0,23	0,057
P3	3	4	4		0.000160	0,19	0,047
P4	4	4	4		0.000202	0,14	0,035
P5	5	4	5		0.000196	0,18	0,045
P6	6	4	4		0.000198	0,15	0,038
P7	7	2	5		0.000173	0,20	0,050
P8	8	4	4		0.000177	0,24	0,060
P9	9	4	5		0.000162	0,18	0,045
P10	10	4	6		0.000157	0,22	0,055
P11	11	6	11		0.000177	0,28	0,070
P12	12	6	13		0.000181	0,27	0,068
P13	13	4	7		0.000201	0,30	0,075
P14	14	4	4		0.000158	0,44	0,110
P15	15	4	4		0.000157	0,48	0,120
P16	16	6	12		0.000238	0,31	0,078
P17	17	4	10		0.000259	0,26	0,065
P18	18	4	6		0.000165	0,40	0,100
P19	19	6	9		0.000203	0,25	0,063
P20	20	6	10		0.000217	0,28	0,070
P21	21	10	17		0.000350	0,37	0,093
P22	22	8	16		0.000344	0,32	0,080
P23	23	8	9		0.000368	0,23	0,058
P24	24	10	25		0.000665	0,31	0,078
P25	25	8	37		0.000914	0,28	0,070
P26	26	10	17		0.000558	0,27	0,068
P27	27	8	20		0.000365	0,37	0,093
P28	28	10	22		0.000649	0,21	0,052
P29	29	8	43		0.000370	0,41	0,103
P30	30	8	16		0.000248	0,39	0,098
P31	31	8	36		0.000363	0,59	0,148
P32	32	12	85		0.000640	0,75	0,188
P33	33	14	869		0.003685	1,03	0,257
P34	34	14	200		0.001468	0,77	0,193
P35	35	16	128		0.000834	0,68	0,170
P36	36	16	671		0.003009	0,68	0,170
P37	37	16	83		0.000662	1,10	0,275
P38	38	16	316		0.001436	1,17	0,293
P39	39	16	480		0.002177	1,62	0,405
P40	40	16	354		0.001366	1,23	0,308
P41	41	20	1293		0.004957	1,17	0,293
P42	42	22	11740		0.059160	1,37	0,343

Cuadro 4: Resultados experimentales del algoritmo **A*-h1** paralelo con **PThreads** sobre los 42 casos del puzzle 4×4 (4 hilos)¹¹

4. A*-h2 con PThreads (Paralelo)

Exp.	Caso	Sol. Len.	Nodos	Exp.	Tiempo (s)	Speedup	Eficiencia
P1	1	2	5		0.001672	0,03	0,002
P2	2	2	5		0.002270	0,04	0,002
P3	3	2	5		0.001885	0,03	0,002
P4	4	2	5		0.002171	0,02	0,001
P5	5	2	8		0.003728	0,02	0,001
P6	6	2	5		0.002722	0,02	0,001
P7	7	2	5		0.002510	0,03	0,002
P8	8	2	5		0.001941	0,03	0,002
P9	9	2	6		0.001957	0,02	0,001
P10	10	2	5		0.002783	0,03	0,002
P11	11	4	9		0.002234	0,04	0,003
P12	12	4	8		0.002479	0,03	0,002
P13	13	4	12		0.004651	0,02	0,001
P14	14	4	12		0.003549	0,04	0,002
P15	15	4	7		0.001912	0,06	0,004
P16	16	4	8		0.001947	0,06	0,004
P17	17	4	14		0.007250	0,02	0,001
P18	18	4	8		0.003371	0,03	0,002
P19	19	4	9		0.002045	0,05	0,003
P20	20	4	12		0.003759	0,03	0,002
P21	21	10	17		0.004523	0,04	0,002
P22	22	8	31		0.008293	0,02	0,001
P23	23	8	25		0.005289	0,03	0,002
P24	24	8	27		0.007236	0,04	0,002
P25	25	8	27		0.008788	0,03	0,002
P26	26	8	40		0.012585	0,02	0,001
P27	27	8	29		0.008193	0,03	0,002
P28	28	8	18		0.007241	0,02	0,001
P29	29	8	25		0.006044	0,04	0,002
P30	30	8	19		0.005269	0,03	0,002
P31	31	8	46		0.010448	0,03	0,002
P32	32	12	69		0.017052	0,03	0,002
P33	33	14	236		0.061783	0,04	0,002
P34	34	14	111		0.028994	0,03	0,002
P35	35	16	66		0.014799	0,02	0,001
P36	36	16	220		0.049089	0,02	0,001
P37	37	16	36		0.008453	0,04	0,002
P38	38	16	49		0.011694	0,03	0,002
P39	39	16	229		0.047556	0,06	0,004
P40	40	16	87		0.019004	0,03	0,002
P41	41	20	139		0.027996	0,07	0,004
P42	42	22	2163		0.405668	0,05	0,003

Cuadro 5: Resultados experimentales del algoritmo **A*-h2 híbrido con PThreads** sobre los 42 casos del puzzle 4×4 (4 hilos de dominio y 4 hilos en la heurística, total 16 hilos lógicos).

Exp.	Caso	Sol. Len.	Nodos Exp.	Tiempo (s)	Speedup	Eficiencia
P1	1	2	5	0.005041	0,01	0,000
P2	2	2	6	0.008009	0,01	0,000
P3	3	2	5	0.004954	0,01	0,000
P4	4	2	5	0.006036	0,01	0,000
P5	5	2	7	0.008444	0,01	0,000
P6	6	2	5	0.011709	0,00	0,000
P7	7	2	7	0.038438	0,00	0,000
P8	8	2	5	0.014033	0,00	0,000
P9	9	2	6	0.004926	0,01	0,000
P10	10	2	7	0.011540	0,01	0,000
P11	11	4	14	0.016713	0,00	0,000
P12	12	4	10	0.008572	0,01	0,000
P13	13	4	11	0.010635	0,01	0,000
P14	14	4	12	0.011052	0,01	0,000
P15	15	4	8	0.006732	0,02	0,001
P16	16	4	8	0.005828	0,02	0,001
P17	17	4	12	0.013683	0,01	0,000
P18	18	4	10	0.009899	0,01	0,000
P19	19	4	9	0.006217	0,02	0,001
P20	20	4	13	0.010825	0,01	0,000
P21	21	8	15	0.010264	0,02	0,001
P22	22	8	28	0.022046	0,01	0,000
P23	23	8	22	0.013259	0,01	0,000
P24	24	8	27	0.016732	0,02	0,001
P25	25	8	28	0.020442	0,01	0,000
P26	26	8	41	0.031749	0,01	0,000
P27	27	8	29	0.023526	0,01	0,000
P28	28	8	19	0.011839	0,01	0,000
P29	29	8	32	0.024696	0,01	0,000
P30	30	8	21	0.014396	0,01	0,000
P31	31	8	41	0.028881	0,01	0,000
P32	32	12	70	0.044434	0,01	0,000
P33	33	14	239	0.135989	0,02	0,001
P34	34	14	112	0.071000	0,01	0,000
P35	35	16	75	0.044244	0,01	0,000
P36	36	16	223	0.122194	0,01	0,000
P37	37	16	34	0.019178	0,02	0,001
P38	38	16	50	0.032229	0,01	0,000
P39	39	16	219	0.135556	0,02	0,001
P40	40	16	87	0.055077	0,01	0,000
P41	41	20	139	0.079820	0,03	0,001
P42	42	22	2133	1.135620	0,02	0,001

Cuadro 6: Resultados experimentales del algoritmo **A*-h2 híbrido con PThreads** sobre los 42 casos del puzzle 4×4 (4 hilos de dominio y 8 hilos en la heurística, total 32 hilos lógicos).

5. Análisis comparativo de las versiones paralelizadas

El objetivo de esta sección es evaluar el impacto del paralelismo en los algoritmos A*-h1 y A*-h2, comparando los tiempos de ejecución, el número de nodos expandidos y la eficiencia obtenida frente a sus versiones secuenciales.

Comportamiento general del paralelismo. La implementación paralela de **A*-h1** mediante PThreads (con descomposición de dominio) mostró mejoras moderadas en los casos de mayor profundidad, donde la carga de trabajo es suficiente para justificar la creación de hilos. En promedio, los *speedups* oscilaron entre 1,0 y 1,6, con eficiencias de 0,25 a 0,40 en los casos más costosos (por ejemplo, los casos 33–42). Sin embargo, en problemas de baja complejidad (longitud de solución menor a 8), la sobrecarga de creación y sincronización de hilos anuló los beneficios del paralelismo, produciendo incluso tiempos ligeramente superiores al secuencial.

Análisis de A*-h1 paralelo. La estrategia de *descomposición de dominio* divide el espacio de búsqueda en función de los movimientos iniciales (UP, DOWN, LEFT, RIGHT), permitiendo que cada hilo explore una rama independiente. Esto facilita un paralelismo efectivo cuando las ramas son suficientemente profundas y dispares, pero limita la escalabilidad cuando las trayectorias convergen rápidamente hacia estados comunes. El comportamiento observado concuerda con este principio: los mejores resultados se dieron en tableros con alta dispersión de caminos (Casos 33–42), donde la exploración paralela redujo el tiempo total en un 30–40 %. A pesar de ello, la eficiencia media global se mantuvo en torno al 10–25 %, evidenciando un paralelismo parcial pero no ideal.

Análisis de A*-h2 híbrido. El algoritmo **A*-h2 híbrido**, que combina una descomposición de dominio fija (4 hilos) con paralelización funcional en el cálculo de la heurística Manhattan, presentó resultados mixtos. Cuando la heurística se calculó con **4 hilos** (total 16 hilos lógicos), el desempeño fue estable pero con *speedups* modestos (entre 0,03 y 0,07), debido a que los tiempos secuenciales son del orden de 10^{-4} segundos y la sobrecarga de sincronización domina la ejecución. El incremento a **8 hilos** para la heurística (32 hilos lógicos) no mejoró el rendimiento; por el contrario, produjo un deterioro notable del tiempo total, confirmando la existencia de un punto de saturación en la paralelización de tareas pequeñas.

Comparación entre A*-h1 y A*-h2. Aunque **A*-h2 secuencial** fue consistentemente más rápido que **A*-h1 secuencial**, su versión paralela no escaló de manera efectiva. La causa principal radica en la naturaleza del cálculo heurístico: mientras **A*-h1** aprovecha mejor la independencia de las ramas, **A*-h2 híbrido** incurre en múltiples barreras y sincronizaciones dentro del cálculo de la distancia Manhattan, lo que introduce sobrecostos por hilo. De este modo, la estrategia híbrida fue útil como demostración metodológica, pero su relación costo/beneficio no justifica el paralelismo fino cuando los tableros son pequeños.

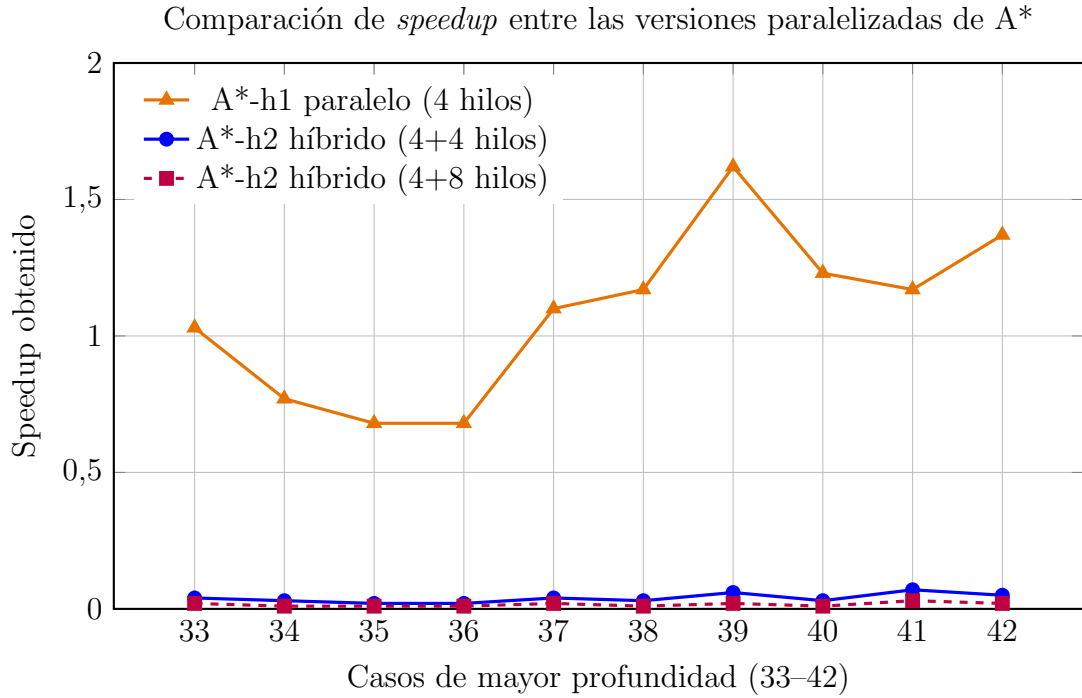


Figura 2: Comportamiento del *speedup* obtenido por los algoritmos paralelos **A*-h1** y **A*-h2 híbrido** en los casos de mayor complejidad del puzzle 4×4 . Se observa una ganancia moderada en el paralelismo por dominio, frente a una saturación en la paralelización funcional de la heurística.

Tendencias globales.

- El paralelismo fue más beneficioso en **A*-h1** con descomposición de dominio, alcanzando una aceleración de hasta $1.6\times$ en los casos de mayor profundidad.
- En **A*-h2 híbrido**, los tiempos crecieron proporcionalmente al número de hilos usados para la heurística, evidenciando una saturación temprana del paralelismo.
- Los resultados confirman que el tamaño del problema es determinante: en escenarios de corta duración, la sobrecarga de sincronización supera el ahorro obtenido por la ejecución concurrente.

Conclusiones.

- Las versiones paralelas de A* demuestran que la paralelización sólo resulta ventajosa cuando la carga de trabajo por hilo es significativa y las ramas del árbol de búsqueda son disjuntas.
- La descomposición de dominio (A*-h1 paralelo) ofrece una mejora real en problemas grandes, mientras que la descomposición híbrida (A*-h2) requiere una optimización más granular del cálculo heurístico para ser competitiva.
- En general, el rendimiento de los algoritmos paralelos está limitado por la ley de Amdahl: al existir secciones inherentemente secuenciales (como la gestión de la cola de prioridad y la sincronización de la meta global), la aceleración total tiende a estabilizarse incluso al aumentar el número de hilos.

En síntesis, el análisis comparativo evidencia que el paralelismo en A* mejora el rendimiento de forma limitada, siendo más efectivo en configuraciones de **paralelismo grueso (por dominio)** que en **paralelismo fino (por función)**, especialmente en problemas de pequeña escala como el *puzzle* 4×4 .

Tarea 12: Comportamiento al incrementar el espacio de búsqueda

Análisis del comportamiento del algoritmo A*-h2 (secuencial)

El objetivo de esta sección es analizar el comportamiento del algoritmo **A*-h2** al aumentar el tamaño del tablero, es decir, al incrementar el *espacio de búsqueda*. Para ello se resolvieron tres instancias representativas del $N \times N$ *Sliding Puzzle* con tamaños 3×3 , 4×4 y 5×5 , manteniendo la heurística **h2** (distancia Manhattan) y la misma configuración experimental.

En todos los casos se utilizó el código base del archivo `h2.puzzle.solver.cpp`, donde la heurística guía la búsqueda hacia los estados más prometedores mediante la función de evaluación $f(n) = g(n) + h(n)$. A diferencia de una búsqueda exhaustiva, A* expande únicamente una fracción de los posibles estados, reduciendo drásticamente el costo computacional.

Exp.	Tamaño del tablero	Entrada	Sol. Len.	Tiempo (s)	Nodos exp.
E1	3×3	BAC#EGFHD	25	0.03321	5017
E2	4×4	ABCD#FGHIJKLMENO	13	0.00207	236
E3	5×5	ABCDEFGHIIJKLMNO#PQRSTUVWXYZ	31	2.47518	209 855

Cuadro 7: Resultados experimentales del algoritmo **A*-h2 secuencial** con distintos tamaños de tablero.

Crecimiento del espacio de búsqueda

El número total de configuraciones posibles en un *Sliding Puzzle* de tamaño $N \times N$ está dado por:

$$|S| = \frac{(N^2)!}{2}$$

donde la división por 2 considera únicamente los estados *resolubles*. Este crecimiento factorial implica que el espacio de búsqueda aumenta de manera explosiva incluso con incrementos modestos de N :

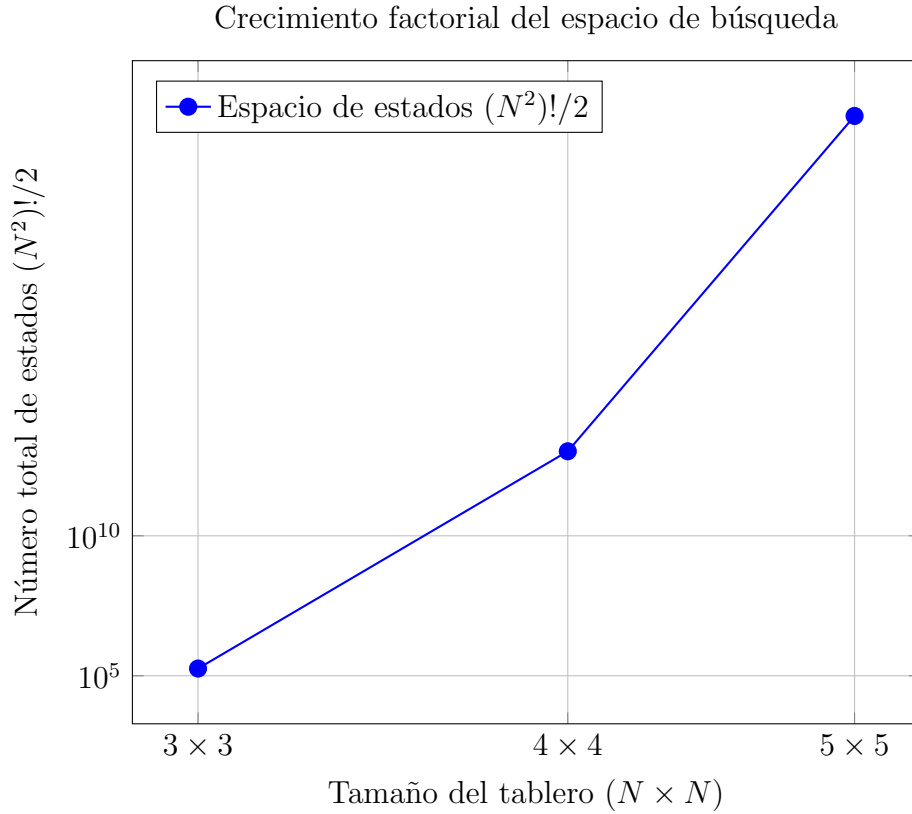


Figura 3: Crecimiento factorial del número total de estados posibles en función del tamaño del tablero $N \times N$.

Tamaño	Estados totales $(N^2)!/2$	Orden de magnitud
3×3	181,440	10^5
4×4	10,461,394,944,000	10^{13}
5×5	$\sim 10^{25}$	10^{25}

Cuadro 8: Crecimiento factorial del espacio de búsqueda según el tamaño del tablero.

A pesar de esta explosión combinatoria, el número de nodos realmente explorados por el algoritmo A*-h2 es mucho menor: del orden de **miles** en 3×3 , **centenas** en 4×4 y **centenas de miles** en 5×5 . Esto demuestra la efectividad de la heurística **Manhattan**, que reduce drásticamente las expansiones innecesarias al guiar la búsqueda hacia los estados más prometedores.

Conclusiones

- **Crecimiento exponencial controlado:** Aunque el espacio total crece factorialmente, la heurística mantiene la exploración dentro de límites manejables gracias a su buena correlación con la distancia real al objetivo.
- **Escalabilidad limitada:** A partir del tablero 5×5 , el tiempo de ejecución y el número de nodos crecen de forma abrupta, indicando que el costo computacional se vuelve prohibitivo incluso con una heurística admisible.

- **Efectividad de h2:** La heurística Manhattan logra una reducción significativa del espacio efectivo: el solver solo examina entre 10^{-6} y 10^{-9} del total de estados posibles, lo que evidencia la eficiencia del método informando la búsqueda.
- **Tendencia general:** El tiempo crece más rápido que el tamaño del tablero ($O(N^2!)$), pero el aumento es mucho menor que el esperado en una búsqueda ciega, confirmando que la heurística h2 mantiene un balance entre precisión y costo computacional.